

- Description

This module manages a smart drone delivery system, ensuring that:

1. The system continuously monitors the drone's current payload (in kilograms) to ensure it does not exceed the maximum payload capacity.
2. If the payload exceeds the maximum capacity, an overload warning is triggered and the drone will not be cleared for flight.
3. The drone's flight speed is automatically adjusted based on the payload: as payload increases, the flight speed is reduced to conserve energy and maintain safety.

This system helps optimize drone delivery efficiency, maintain flight safety, and ensure energy-efficient operation.

- Specification

module SmartDronePayloadControlEx

var currentPayload, maxPayload, baseFlightSpeed, currentFlightSpeed: real

process updatePayload(newPayload: real) returns (currentPayload: real, currentFlightSpeed: real)

functional scenario form:

currentPayload <= maxPayload && currentFlightSpeed = baseFlightSpeed * (1 - 0.5 * (currentPayload / maxPayload)), Flight speed is adjusted according to payload.

||

currentPayload > maxPayload && currentFlightSpeed = 0, Overload warning triggered, flight speed set to 0.

end process

end module

- Java Code

```
package code;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
/**
 * c126_SmartDroneDeliveryEx
 * javac code/c126_SmartDroneDeliveryEx.java
 * java code.c126_SmartDroneDeliveryEx
 *
 * java code.c126_SmartDroneDeliveryEx MODE=VT LOG=on
 */
public class c126_SmartDroneDeliveryEx {
    /* =====
     * Alerts / Logger / Stats
     * ===== */
    enum AlertType {
```

```

        INFO, WARN, ERROR,
        PAYLOAD_OVER, PAYLOAD_NEG, SPEED_ZERO, WEATHER_SEVERE,
        NOFLY_CONFLICT, ROUTE_BLOCKED, BATTERY_LOW, SENSOR_FAULT,
        SCHED_ENQ, SCHED_DISPATCH, PLANNER_REPLAN, COLLISION_RISK
    }
    static final class Alert {
        final AlertType type;
        final String msg;
        final long ts;
        Alert(AlertType t, String m) { this.type = t; this.msg = m; this
s.ts = System.currentTimeMillis(); }
        @Override public String toString() { return "[" + type + "]" + " +
msg + " @" + ts; }
    }
    static final class SystemLogger {
        final List<String> lines = new ArrayList<>();
        final List<Alert> alerts = new ArrayList<>();
        boolean console = true;
        void setConsole(boolean on) { console = on; }
        void log(String s) { lines.add(s); if (console) System.out.prin
tln(s); }
        void alert(AlertType t, String m) {
            Alert a = new Alert(t, m);
            alerts.add(a);
            if (console) System.out.println("ALERT: " + a);
        }
        void summary() {
            log("\n=== Logger Summary ===");
            log("Lines=" + lines.size() + ", Alerts=" + alerts.size());
            Map<AlertType, Long> cnt = new EnumMap<>(AlertType.class);
            for (Alert a : alerts) cnt.put(a.type, cnt.getDefault(a.t
ype, 0L) + 1);
            for (AlertType t : AlertType.values()) log(" - " + t + ": "
+ cnt.getDefault(t, 0L));
        }
    }
    static final class Stats {
        long payloadUpdates, denied, replan, schedEnq, schedDispatch, r
outeBlocks, collisions;
        long lowBattery, severeWeather;
        void print(SystemLogger L) {
            L.log("\n=== Drone Stats ===");
            L.log("PayloadUpdates=" + payloadUpdates + ", Denied=" + de
nied + ", Replan=" + replan);

```

```

        L.log("Scheduler enq=" + schedEnq + ", dispatch=" + schedDi
spatch);
        L.log("RouteBlocks=" + routeBlocks + ", Collisions=" + coll
isions);
        L.log("LowBattery=" + lowBattery + ", SevereWeather=" + sev
ereWeather);
    }
}
/* =====
 * Environment: Weather / Airspace / Grid
 * ===== */
enum WeatherType { CLEAR, WINDY, RAIN, STORM }
static final class Weather {
    final WeatherType type;
    final double windKmh;
    final double rain;    // 0..1
    final boolean lightning;
    Weather(WeatherType type, double windKmh, double rain, boolean
lightning) {
        this.type = type; this.windKmh = windKmh; this.rain = rain;
this.lightning = lightning;
    }
    static Weather random(Random r) {
        int t = r.nextInt(100);
        if (t < 60) return new Weather(WeatherType.CLEAR, r.nextDou
ble()*10 - 5, 0.0, false);
        if (t < 80) return new Weather(WeatherType.WINDY, r.nextDou
ble()*30 - 15, 0.0, false);
        if (t < 95) return new Weather(WeatherType.RAIN, r.nextDoub
le()*12 - 6, r.nextDouble()*0.8, false);
        return new Weather(WeatherType.STORM, r.nextDouble()*50 - 2
5, r.nextDouble(), r.nextBoolean());
    }
    boolean severe() { return type == WeatherType.STORM || rain > 0
.7 || Math.abs(windKmh) > 35; }
}
static final class Airspace {
    final int width, height, levels; // x,y,z
    final boolean[][][] nofly;    // true
    final Map<Long, Set<String>> slotOccupancy = new ConcurrentHash
Map<>();
    final SystemLogger L;
    final Stats stats;
    Airspace(int w, int h, int z, SystemLogger L, Stats stats) {

```

```

        this.width = Math.max(10, w);
        this.height = Math.max(10, h);
        this.levels = Math.max(1, z);
        this.nofly = new boolean[levels][height][width];
        this.L = L; this.stats = stats;
    }
    boolean inBounds(int x, int y, int z) {
        return x>=0 && x<width && y>=0 && y<height && z>=0 && z<lev
els;
    }
    void addNoFlyRect(int z, int x0, int y0, int x1, int y1) {
        int xx0 = Math.max(0, Math.min(x0, x1));
        int xx1 = Math.min(width-1, Math.max(x0, x1));
        int yy0 = Math.max(0, Math.min(y0, y1));
        int yy1 = Math.min(height-1, Math.max(y0, y1));
        z = Math.max(0, Math.min(levels-1, z));
        for (int y = yy0; y <= yy1; y++)
            for (int x = xx0; x <= xx1; x++)
                nofly[z][y][x] = true;
    }
    boolean isNoFly(int x, int y, int z) {
        if (!inBounds(x,y,z)) return true;
        return nofly[z][y][x];
    }
    boolean reserve(long t, int x, int y, int z, String id) {
        long key = (t<<20) ^ (x<<10) ^ (y<<5) ^ z;
        slotOccupancy.computeIfAbsent(key, k->Collections.synchroni
zedSet(new HashSet<>()));
        Set<String> set = slotOccupancy.get(key);
        synchronized (set) {
            if (!set.isEmpty()) {
                stats.collisions++;
                L.alert(AlertType.COLLISION_RISK, "t="+t+" @"+"x+",
"+y+", "+z+"");
                return false;
            }
            set.add(id);
            return true;
        }
    }
}
/* =====
* Battery / Payload / Sensors
* ===== */

```

```

static final class Battery {
    double level; // 0..100 %
    final double idleDrainPerTick;
    final double perKgPerKmDrain;
    Battery(double init, double idle, double perKgKm) {
        this.level = clamp01(init);
        this.idleDrainPerTick = Math.max(0, idle);
        this.perKgPerKmDrain = Math.max(0, perKgKm);
    }
    void drainIdle() { level = clamp01(level - idleDrainPerTick); }
    void drainMove(double km, double payloadKg) {
        double d = km * (0.03 + perKgPerKmDrain * Math.max(0, payloadKg));
        level = clamp01(level - d);
    }
    void rechargeTo(double v) { level = clamp01(v); }
    boolean low() { return level < 15.0; }
    static double clamp01(double v) { return Math.max(0, Math.min(100, v)); }
}

static final class PayloadBay {
    final double maxPayload;
    double currentPayload;
    PayloadBay(double max, double init) { this.maxPayload = max; this.currentPayload = init; }
    boolean over() { return currentPayload > maxPayload; }
    boolean negative() { return currentPayload < 0; }
    double ratio() { return (maxPayload <= 0)? 1.0 : (currentPayload / maxPayload); }
    void set(double v) { currentPayload = v; }
}

interface Sensor<T> { T read(); }

static final class PayloadSensor implements Sensor<Double> {
    final PayloadBay bay;
    PayloadSensor(PayloadBay b) { this.bay = b; }
    public Double read() { return bay.currentPayload; }
}

static final class BatterySensor implements Sensor<Double> {
    final Battery bat;
    BatterySensor(Battery b) { this.bat = b; }
    public Double read() { return bat.level; }
}

static final class SpeedSensor implements Sensor<Double> {
    final DroneCore core;

```

```

        SpeedSensor(DroneCore c) { this.core = c; }
        public Double read() { return core.currentFlightSpeed; }
    }
    static final class WindSensor implements Sensor<Double> {
        final Random r;
        WindSensor(long seed) { r = new Random(seed); }
        public Double read() { return r.nextDouble()*20 - 10; }
    }
    /* =====
    * Planner / Scheduler
    * ===== */
    static final class Waypoint {
        final int x,y,z;
        Waypoint(int x,int y,int z){this.x=x;this.y=y;this.z=z;}
        @Override public String toString(){return "("+x+","+y+","+z+")"}
    };
    }
    static final class Route {
        final List<Waypoint> steps = new ArrayList<>();
        double totalKm; // 简化: 每步1单位=0.1km
        void add(Waypoint w){ steps.add(w); totalKm += 0.1; }
        boolean empty(){ return steps.isEmpty(); }
    }
    static final class Planner {
        final Airspace air;
        final SystemLogger L;
        final Stats stats;
        Planner(Airspace air, SystemLogger L, Stats stats){this.air=air
;this.L=L;this.stats=stats;}
        Route plan(Waypoint s, Waypoint t) {
            if (!air.inBounds(s.x, s.y, s.z) || !air.inBounds(t.x, t.y,
t.z)) {
                stats.routeBlocks++;
                L.alert(AlertType.ROUTE_BLOCKED, "start or target OOB")
;
                return new Route();
            }
            int[] dx = {1,-1,0,0,0,0};
            int[] dy = {0,0,1,-1,0,0};
            int[] dz = {0,0,0,0,1,-1};
            boolean[][][] vis = new boolean[air.levels][air.height][air
.width];
            int[][][] px = new int[air.levels][air.height][air.width];
            int[][][] py = new int[air.levels][air.height][air.width];

```

```

        int[][][] pz = new int[air.levels][air.height][air.width];
        Deque<int[]> q = new ArrayDeque<>();
        q.add(new int[]{s.x,s.y,s.z}); vis[s.z][s.y][s.x]=true;
        boolean found=false;
        while(!q.isEmpty()){
            int[] cur=q.poll();
            if(cur[0]==t.x && cur[1]==t.y && cur[2]==t.z){found=true
e;break;}
            for(int i=0;i<6;i++){
                int nx=cur[0]+dx[i], ny=cur[1]+dy[i], nz=cur[2]+dz[
i];
                if(air.inBounds(nx,ny,nz) && !vis[nz][ny][nx] && !a
ir.isNoFly(nx,ny,nz)){
                    vis[nz][ny][nx]=true;
                    px[nz][ny][nx]=cur[0]; py[nz][ny][nx]=cur[1]; p
z[nz][ny][nx]=cur[2];
                    q.add(new int[]{nx,ny,nz});
                }
            }
        }
        Route r = new Route();
        if(!found){
            stats.routeBlocks++;
            L.alert(AlertType.ROUTE_BLOCKED, "no path");
            return r;
        }
        int cx=t.x, cy=t.y, cz=t.z;
        List<Waypoint> rev = new ArrayList<>();
        while(!(cx==s.x && cy==s.y && cz==s.z)){
            rev.add(new Waypoint(cx,cy,cz));
            int tx=px[cz][cy][cx], ty=py[cz][cy][cx], tz=pz[cz][cy]
[cx];
            cx=tx;cy=ty;cz=tz;
        }
        Collections.reverse(rev);
        for(Waypoint w:rev) r.add(w);
        return r;
    }
    List<long[]> reserve(Route route, long start, String id) {
        List<long[]> sched = new ArrayList<>();
        long t = start;
        for (Waypoint w : route.steps) {
            boolean ok = air.reserve(t, w.x, w.y, w.z, id);
            if (!ok) {

```

```

        stats.replan++;
        L.alert(AlertType.PLANNER_REPLAN, "delay 1 tick @"
+ w);

        t++;
        air.reserve(t, w.x, w.y, w.z, id);
    }
    sched.add(new long[]{t, w.x, w.y, w.z});
    t++;
}
return sched;
}
}

enum TaskType { MOVE, CHARGE, HOVER }
static final class Task {
    final TaskType type;
    final Waypoint dest;    // MOVE
    final int priority;
    final String tag;

    Task(TaskType type, Waypoint d, int p, String tag){this.type=ty
pe;this.dest=d;this.priority=p;this.tag=tag;}

    @Override public String toString(){return type + (dest==null?"
:"->" + dest) + "#p" + priority + (tag==null?"":"[" + tag + "]);}
}

static final class Scheduler {
    final PriorityQueue<Task> pq;
    final SystemLogger L; final Stats stats;
    Scheduler(SystemLogger L, Stats s){
        this.L=L; this.stats=s;
        this.pq = new PriorityQueue<>(Comparator
            .comparingInt((Task t)->t.priority)
            .thenComparing(t->t.tag==null?"":t.tag));
    }

    void enqueue(Task t){ pq.offer(t); stats.schedEnq++; L.alert(Al
ertType.SCHED_ENQ, "task="+t); }

    Task dispatch(){ Task t=pq.poll(); if(t!=null){stats.schedDispa
tch++; L.alert(AlertType.SCHED_DISPATCH, "task="+t);} return t; }

    boolean isEmpty(){ return pq.isEmpty(); }
}

static final class DroneCore {
    double maxPayload;
    double currentPayload;
    double baseFlightSpeed;
    double currentFlightSpeed;
    final String id;

```

```

    final SystemLogger L;
    final Stats stats;
    final Battery battery;
    final PayloadBay bay;
    final PayloadSensor payloadSensor;
    final BatterySensor batterySensor;
    final SpeedSensor speedSensor;
    final WindSensor windSensor;
    final Planner planner;
    final Scheduler scheduler;
    final Airspace air;
    final Random rand;
    Waypoint pos;
    final Deque<Task> queue = new ArrayDeque<>();
    DroneCore(String id,
                double maxPayload, double baseFlightSpeed, double ini
tialPayload,
                Airspace air, Planner planner, Scheduler scheduler,
                SystemLogger L, Stats stats, long seed) {
        this.maxPayload = maxPayload;
        this.baseFlightSpeed = baseFlightSpeed;
        this.currentPayload = initialPayload;
        this.currentFlightSpeed = baseFlightSpeed;
        this.id = id;
        this.L = L;
        this.stats = stats;
        this.battery = new Battery(100, 0.05, 0.004);
        this.bay = new PayloadBay(maxPayload, initialPayload);
        this.payloadSensor = new PayloadSensor(bay);
        this.batterySensor = new BatterySensor(battery);
        this.speedSensor = new SpeedSensor(this);
        this.windSensor = new WindSensor(seed^0xD0D0L);
        this.planner = planner;
        this.scheduler = scheduler;
        this.air = air;
        this.rand = new Random(seed);
        this.pos = new Waypoint(rand.nextInt(Math.max(1,air.width))
, rand.nextInt(Math.max(1,air.height)), rand.nextInt(Math.max(1,air.lev
els)));
        adjustFlightSpeed();
    }
    void updatePayload(double newPayload) {
        stats.payloadUpdates++;
        bay.set(newPayload);
    }

```

```

        currentPayload = bay.currentPayload;
        if (currentPayload > maxPayload) {
            System.out.println("WARNING: Payload exceeds maximum ca
capacity! Flight clearance denied.");
            currentFlightSpeed = 0;
            stats.denied++;
            scheduler.enqueue(new Task(TaskType.CHARGE, null, 0, "a
uto-denied-charge"));
            return;
        } else {
            adjustFlightSpeed();
            System.out.printf(Locale.ROOT, "Payload updated to: %.2
f kg, Adjusted Flight Speed: %.2f km/h%n", currentPayload, currentFligh
tSpeed);
        }
        if (bay.negative()) {
            L.alert(AlertType.PAYLOAD_NEG, "negative payload set: "
+ currentPayload + " kg");
        }
        if (currentFlightSpeed == 0) {
            L.alert(AlertType.SPEED_ZERO, "speed=0 after payload up
date");
        }
    }
    void adjustFlightSpeed() {
        if (currentPayload <= maxPayload) {
            double loadFactor = (maxPayload<=0) ? 1.0 : (currentPay
load / maxPayload);
            currentFlightSpeed = baseFlightSpeed * (1 - 0.5 * loadF
actor);
        }
    }
    void execute(Route r, List<long[]> sched) {
        for (long[] tp : sched) {
            long t = tp[0];
            int x = (int)tp[1], y = (int)tp[2], z = (int)tp[3];
            // idle drain per tick
            battery.drainIdle();
            double km = 0.1;
            battery.drainMove(km, Math.max(0, currentPayload));
            if (rand.nextDouble()<0.05) {
                Weather w = Weather.random(rand);
                if (w.severe()) {
                    stats.severeWeather++;
                }
            }
        }
    }

```

```

        L.alert(AlertType.WEATHER_SEVERE, "storm-
ish @t="+t);
    }
}
double wind = windSensor.read();
pos = new Waypoint(x,y,z);
if (battery.low()) {
    stats.lowBattery++;
    L.alert(AlertType.BATTERY_LOW, "level=" + String.fo
rmat(Locale.ROOT, "%.1f", battery.level));
    scheduler.enqueue(new Task(TaskType.CHARGE, null, 0
, "auto-low-charge"));
}
}
}
void enqueue(Task t) { queue.addLast(t); }
void tick(long globalTime) {
    battery.drainIdle();
    if (queue.isEmpty()) return;
    Task task = queue.peekFirst();
    if (task.type == TaskType.CHARGE) {
        battery.rechargeTo(100);
        L.log("Drone "+id+" charged to 100% at "+pos+" via "+ta
sk);
        queue.pollFirst();
        return;
    }
    if (task.type == TaskType.HOVER) {
        battery.drainMove(0.01, Math.max(0, currentPayload));
        queue.pollFirst();
        return;
    }
    if (task.type == TaskType.MOVE) {
        if (task.dest == null) { queue.pollFirst(); return; }
        Route route = planner.plan(pos, task.dest);
        if (route.empty()) { stats.routeBlocks++; queue.pollFir
st(); return; }
        List<long[]> schedule = planner.reserve(route, globalTi
me, id);
        execute(route, schedule);
        queue.pollFirst();
    }
}
}

```

```

        public double getCurrentFlightSpeed() { return currentFlightSpeed; }
    }
    public double getCurrentPayload() { return currentPayload; }
}
public static class c86_SmartDroneDelivery {
    private final DroneCore core;
    public c86_SmartDroneDelivery(double maxPayload, double baseFlightSpeed, double initialPayload) {
        SystemLogger L = new SystemLogger();
        Stats stats = new Stats();
        Airspace air = new Airspace(40,40,3, L, stats);
        air.addNoFlyRect(1, 10,10, 15,15);
        Planner planner = new Planner(air, L, stats);
        Scheduler sched = new Scheduler(L, stats);
        this.core = new DroneCore("D1", maxPayload, baseFlightSpeed, initialPayload, air, planner, sched, L, stats, 42L);
    }
    public void updatePayload(double newPayload) { core.updatePayload(newPayload); }
    public double getCurrentFlightSpeed() { return core.getCurrentFlightSpeed(); }
    public double getCurrentPayload() { return core.getCurrentPayload(); }
}
/* =====
 * Scenarios & Runner
 * ===== */
enum Mode { VT, FT, Z3, UVT }
static final class Scenario {
    final String name;
    final List<Runnable> steps = new ArrayList<>();
    Scenario(String n){this.name=n;}
    Scenario step(Runnable r){ steps.add(r); return this; }
    void run(SystemLogger L){
        L.log("\n=== Scenario: "+name+" ===");
        int i=0; for(Runnable r: steps){ r.run(); i++; }
        L.log("Steps executed: "+i);
    }
}
static final class ScenarioFactory {
    static Scenario VT(c86_SmartDroneDelivery drone) {
        Scenario s = new Scenario("VT");
        double[] seq = {

```

```

        -55.0, -71.9, -41.7, 81.3, -37.2, 94.2, -
59.9, 54.6, 77.8, -78.5, 27.1, 37.1, 32.6, -45.0, -54.5, -50.6,
        1.2, -58.2, 25.4, 39.7, 85.6, -51.2, -94.3, -56.5, -
42.8, 20.5, 41.8, 30.4, -66.9, 3.2, 98.6, 61.0, 19.3,
        -67.0, 36.2, -77.5, 3.6, 6.2, 17.8, -15.5, -62.1, -
33.7, 83.1, 30.8, 13.6, -11.3, 61.3, -92.8, -24.3,
        -47.8, 70.0, 8.1, -27.5, -
96.8, 90.5, 53.9, 72.7, 5.2, -30.2, -27.8, -15.6, 41.2, -80.8, 51.5, -
74.8,
        46.6, -18.4, -2.5, 35.2, 98.2, 9.4, -43.2, 51.5, -
38.9, 44.0, -83.4, -87.3, -14.3, 76.9, -8.2, -32.8,
        -67.6, -90.7, 92.6, 48.5, 84.5, -32.2, -3.6, 63.8, -
80.6, 75.0, 56.9, 26.7, -25.2, -78.4, -63.6, -5.5,
        49.6, -2.6, 63.4
    };
    for(double v: seq) s.step(()->drone.updatePayload(v));
    return s;
}
static Scenario Z3(c86_SmartDroneDelivery d){
    Scenario s = new Scenario("Z3");
    double[] seq = {
        3,
        3,
        7,
        8,
        81,
        44,
        36,
        56,
        29,
        16,
        21,
        96,
        77,
        70,
        57,
        89,
        22,
        17,
        10,
        67,
        87,
        52,
        70,

```

77,

95,

85,

53,

73,

93,

29,

62,

73,

12,

27,

69,

38,

17,

49,

84,

62,

41,

76,

33,

50,

87,

35,

57,

84,

62,

72,

14,

48,

51,

73,

36,

34,

10,

71,

13,

43,

63,

95,

91,

49,

8,

30,

86,

```

        44,
        66,
        94,
        46,
        35,
        50,
        90,
        13,
        44,
        65,
        72,
        95,
        18,
        44,
        38,
        34,
        47,
        12,
        48,
        86,
        95,
        53,
        29,
        60,
        16,
        70,
        47,
        66,
        44,
        26,
        33,
        65
    };
    for(double v: seq) s.step()->d.updatePayload(v));
    return s;
}
static Scenario UVT(c86_SmartDroneDelivery d){
    Scenario s = new Scenario("UVT");
    double[] seq = {
        3,
        3,
        7,
        8,
        81,

```

44,

36,

56,

29,

16,

21,

96,

77,

70,

57,

89,

22,

17,

10,

67,

87,

52,

70,

77,

95,

85,

53,

73,

93,

29,

62,

73,

12,

27,

69,

38,

17,

49,

84,

62,

41,

76,

33,

50,

87,

35,

57,

84,

62,

72,

14,

48,

51,

73,

36,

34,

10,

71,

13,

43,

63,

95,

91,

49,

8,

30,

86,

44,

66,

94,

46,

35,

50,

90,

13,

44,

65,

72,

95,

18,

44,

38,

34,

47,

12,

48,

86,

95,

53,

29,

60,

16,

70,

0,
0,
0,
0,
0,
-151587082,
-151587082,
-151587082,
-151587082,
-151587082,
-151587082,
0,
0,
0,
0,
0,
0,
16,
0,
16,
0,
0,
0,
0,
0,
0,
63,
0,
1414812756,
1414812756,
1414812756,
1414812756,
0,
0,
13041664,
0,
-1,
-1,
-254,
0,
-254,
0,
0,
0,
0,

```

        0,
        0,
        0,
        0,
        0,
        0,
        0,
        8,
        0,
        10,
        0,
        157,
        0,
        58,
        0,
        24634,
        0,
        1,
        0,
        120,
        0,
        120,
        0,
        2013265920,
        0
    };
    for(double v: seq) s.step()->d.updatePayload(v));
    return s;
}
}
static final class Runner {
    final SystemLogger L;
    Runner(SystemLogger L){this.L=L;}
    void run(Scenario s){ s.run(L); }
}
public static void main(String[] args) {
    // CLI
    Mode mode = Mode.UVT;
    boolean console = true;
    long seed = 20251104L;
    for(String a: args){
        String x=a.trim();
        if(x.startsWith("MODE=")) {

```

```

        try { mode = Mode.valueOf(x.substring(5).toUpperCase(Locale.ROOT)); } catch (Exception ignore) {}
        } else if (x.startsWith("LOG=")) {
            String v=x.substring(4).toLowerCase(Locale.ROOT);
            console = v.equals("on") || v.equals("true") || v.equals("1");
        } else if (x.startsWith("SEED=")) {
            try { seed = Long.parseLong(x.substring(5)); } catch (Exception ignore) {}
        }
    }
    SystemLogger L = new SystemLogger();
    L.setConsole(console);
    Stats stats = new Stats();
    Airspace air = new Airspace(60,60,4, L, stats);
    air.addNoFlyRect(2, 20,20, 30,30);
    Planner planner = new Planner(air, L, stats);
    Scheduler scheduler = new Scheduler(L, stats);
    c86_SmartDroneDelivery drone = new c86_SmartDroneDelivery(10.0, 60.0, 3.0);
    DroneCore ext = new DroneCore("DX", 12.0, 65.0, 2.0, air, planner, scheduler, L, stats, seed);
    scheduler.enqueue(new Task(TaskType.MOVE, new Waypoint(5,5,1), 5, "warmup"));
    scheduler.enqueue(new Task(TaskType.MOVE, new Waypoint(10,10,1), 5, "stage"));
    scheduler.enqueue(new Task(TaskType.HOVER, null, 9, "hover"));
    Task t;
    while((t=scheduler.dispatch())!=null) ext.enqueue(t);
    AtomicLong clock = new AtomicLong(0);
    for(int i=0;i<8;i++) ext.tick(clock.getAndIncrement());
    Runner runner = new Runner(L);
    Scenario sc;
    switch (mode) {
        case VT:
            sc = ScenarioFactory.VT(drone);
            break;

        case FT:
            sc = ScenarioFactory.FT(drone);
            break;

        case Z3:
            sc = ScenarioFactory.Z3(drone);

```

```

        break;

        default:
            sc = ScenarioFactory.UVT(drone);
        }
        L.log("=== SmartDroneDeliveryEx (Mode="+mode+", Seed="+seed+"
===");
        runner.run(sc);
        stats.print(L);
        L.summary();
    }
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 70%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c126_SmartDroneDeliveryEx.java	62.8 %	93	55	148
▼ c126_SmartDroneDeliveryEx	27.3 %	6	16	22
main(String[])	27.3 %	6	16	22
> PayloadBay	33.3 %	2	4	6
> Weather	33.3 %	4	8	12
> Battery	50.0 %	1	1	2
> ScenarioFactory	50.0 %	2	2	4
> DroneCore	70.0 %	21	9	30
> Scheduler	75.0 %	3	1	4
> SystemLogger	75.0 %	6	2	8
> Task	75.0 %	3	1	4
> Planner	76.5 %	26	8	34
> Airspace	85.0 %	17	3	20
> Scenario	100.0 %	2	0	2
> Alert		0	0	0
> AlertType		0	0	0
> BatterySensor		0	0	0

3. Vibration testing instruction coverage: 79.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c126_SmartDroneDeliveryEx.java	84.0 %	2,502	477	2,979
▼ c126_SmartDroneDeliveryEx	69.1 %	168	75	243
> PayloadBay	48.9 %	22	23	45
> Weather	50.0 %	52	52	104
> BatterySensor	54.5 %	6	5	11
> PayloadSensor	54.5 %	6	5	11
> SpeedSensor	54.5 %	6	5	11
main(String[])	70.0 %	168	72	240
> DroneCore	79.4 %	375	97	472
> ScenarioFactory	80.0 %	439	110	549
> Planner	87.9 %	437	60	497
> c86_SmartDroneDelivery	88.2 %	60	8	68
> Battery	89.2 %	58	7	65
> Airspace	89.6 %	189	22	211
> Scheduler	92.1 %	70	6	76
> Task	94.9 %	37	2	39
> Alert	100.0 %	21	0	21

4. Fuzz testing branch coverage: 70%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c126_SmartDroneDeliveryEx.java	61.2 %	93	59	152
▼ c126_SmartDroneDeliveryEx	27.3 %	6	16	22
> ScenarioFactory	25.0 %	2	6	8
> main(String[])	27.3 %	6	16	22
> PayloadBay	33.3 %	2	4	6
> Weather	33.3 %	4	8	12
> Battery	50.0 %	1	1	2
▼ DroneCore	70.0 %	21	9	30
▲ adjustFlightSpeed()	50.0 %	2	2	4
▲ tick(long)	66.7 %	8	4	12
▲ execute(Route, List<long[]>)	75.0 %	6	2	8
▲ updatePayload(double)	83.3 %	5	1	6
▲ DroneCore(String, double, double)		0	0	0
▲ enqueue(Task)		0	0	0
> Scheduler	75.0 %	3	1	4
> SystemLogger	75.0 %	6	2	8
> Task	75.0 %	3	1	4

5. Fuzz testing instruction coverage: 79.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c126_SmartDroneDeliveryEx.java	59.9 %	2,499	1,670	4,169
▼ c126_SmartDroneDeliveryEx	69.1 %	168	75	243
> ScenarioFactory	25.1 %	436	1,303	1,739
> PayloadBay	48.9 %	22	23	45
> Weather	50.0 %	52	52	104
> BatterySensor	54.5 %	6	5	11
> PayloadSensor	54.5 %	6	5	11
> SpeedSensor	54.5 %	6	5	11
> main(String[])	70.0 %	168	72	240
▼ DroneCore	79.4 %	375	97	472
▲ execute(Route, List<long[]>)	61.2 %	74	47	121
▲ tick(long)	65.1 %	69	37	106
▲ adjustFlightSpeed()	93.3 %	28	2	30
▲ updatePayload(double)	94.4 %	84	5	89
▲ DroneCore(String, double, double)	100.0 %	115	0	115
▲ enqueue(Task)	100.0 %	5	0	5

6. Z3 branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c126_SmartDroneDeliveryEx.java	59.9 %	91	61	152
▼ c126_SmartDroneDeliveryEx	27.3 %	6	16	22
> PayloadBay	16.7 %	1	5	6
> ScenarioFactory	25.0 %	2	6	8
> main(String[])	27.3 %	6	16	22
> Weather	33.3 %	4	8	12
> Battery	50.0 %	1	1	2
> DroneCore	66.7 %	20	10	30
> Scheduler	75.0 %	3	1	4
> SystemLogger	75.0 %	6	2	8
> Task	75.0 %	3	1	4
> Planner	76.5 %	26	8	34
> Airspace	85.0 %	17	3	20
> Scenario	100.0 %	2	0	2
> Alert		0	0	0
> AlertType		0	0	0
> BatterySensor		0	0	0

7. Z3 instruction coverage: 78%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c126_SmartDroneDeliveryEx.java		59.6 %	2,486	1,683	4,169
▼ c126_SmartDroneDeliveryEx		69.1 %	168	75	243
> ScenarioFactory		24.8 %	432	1,307	1,739
> PayloadBay		44.4 %	20	25	45
> Weather		50.0 %	52	52	104
> BatterySensor		54.5 %	6	5	11
> PayloadSensor		54.5 %	6	5	11
> SpeedSensor		54.5 %	6	5	11
> main(String[])		70.0 %	168	72	240
> DroneCore		78.0 %	368	104	472
> Planner		87.9 %	437	60	497
> c86_SmartDroneDelivery		88.2 %	60	8	68
> Battery		89.2 %	58	7	65
> Airspace		89.6 %	189	22	211
> Scheduler		92.1 %	70	6	76
> Task		94.9 %	37	2	39
> Alert		100.0 %	21	0	21

8. Unimproved V-method branch coverage: 70%

Element		Coverage	Covered Branches	Missed Branches	Total Branches
▼ c126_SmartDroneDeliveryEx		27.3 %	6	16	22
> main(String[])		27.3 %	6	16	22
> PayloadBay		33.3 %	2	4	6
> Weather		33.3 %	4	8	12
> Battery		50.0 %	1	1	2
> ScenarioFactory		50.0 %	2	2	4
> DroneCore		70.0 %	21	9	30
> Scheduler		75.0 %	3	1	4
> SystemLogger		75.0 %	6	2	8
> Task		75.0 %	3	1	4
> Planner		76.5 %	26	8	34
> Airspace		85.0 %	17	3	20
> Scenario		100.0 %	2	0	2

9. Unimproved V-method instruction coverage: 79.4%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c126_SmartDroneDeliveryEx		69.1 %	168	75	243
> PayloadBay		48.9 %	22	23	45
> Weather		50.0 %	52	52	104
> BatterySensor		54.5 %	6	5	11
> PayloadSensor		54.5 %	6	5	11
> SpeedSensor		54.5 %	6	5	11
> main(String[])		70.0 %	168	72	240
> DroneCore		79.4 %	375	97	472
> ScenarioFactory		80.0 %	439	110	549
> Planner		87.9 %	437	60	497
> c86_SmartDroneDelivery		88.2 %	60	8	68
> Battery		89.2 %	58	7	65
> Airspace		89.6 %	189	22	211
> Scheduler		92.1 %	70	6	76
> Task		94.9 %	37	2	39
> Alert		100.0 %	21	0	21